

DEVOPS CAREER SERIES

Top 100 DevOps Engineer Interview Questions & Answers

2026 Edition — Elaborated, Interview-Ready Answers

PREPARED BY

Arvind Verma

DevOps Engineer | AWS · Azure · GCP | Kubernetes & IaC Automation

100

INTERVIEW QUESTIONS

20

TOPIC MODULES

1st

PERSON ANSWERS

2026

EDITION

What's inside:

- 1. DevOps Fundamentals
- 2. Version Control & Git
- 3. CI/CD (Jenkins, GitLab CI/CD, GitHub Actions)
- 4. Configuration Management (Ansible, Puppet, Chef)
- 5. Containers & Orchestration (Docker, Kubernetes)
- 6. Cloud & Infrastructure as Code (AWS, Azure, GCP, Terraform)
- 7. Monitoring & Logging (Prometheus, Grafana, ELK Stack)
- 8. Security in DevOps (DevSecOps)
- 9. SRE & Reliability Engineering
- 10. Advanced CI/CD Concepts
- 11. GitOps & Infrastructure Automation
- 12. Kubernetes Advanced Topics
- 13. Monitoring & Logging Advanced Concepts
- 14. Security Best Practices in DevOps
- 15. Cloud-Native & Serverless
- 16. Advanced Networking & Security
- 17. DevOps Culture & Processes
- 18. Performance Optimization
- 19. Incident Management & Disaster Recovery
- 20. Miscellaneous & Future Trends

Table of Contents

1. DevOps Fundamentals	Q1–Q5
2. Version Control & Git	Q6–Q10
3. CI/CD (Jenkins, GitLab CI/CD, GitHub Actions)	Q11–Q15
4. Configuration Management (Ansible, Puppet, Chef)	Q16–Q20
5. Containers & Orchestration (Docker, Kubernetes)	Q21–Q25
6. Cloud & Infrastructure as Code (AWS, Azure, GCP, Terraform)	Q26–Q30
7. Monitoring & Logging (Prometheus, Grafana, ELK Stack)	Q31–Q35
8. Security in DevOps (DevSecOps)	Q36–Q40
9. SRE & Reliability Engineering	Q41–Q45
10. Advanced CI/CD Concepts	Q46–Q50
11. GitOps & Infrastructure Automation	Q51–Q55
12. Kubernetes Advanced Topics	Q56–Q60
13. Monitoring & Logging Advanced Concepts	Q61–Q65
14. Security Best Practices in DevOps	Q66–Q70
15. Cloud-Native & Serverless	Q71–Q75
16. Advanced Networking & Security	Q76–Q80
17. DevOps Culture & Processes	Q81–Q85
18. Performance Optimization	Q86–Q90
19. Incident Management & Disaster Recovery	Q91–Q95

20. Miscellaneous & Future Trends

Q96–Q100

Each module below groups related questions the way an interviewer typically sequences them — fundamentals first, then tooling, then advanced and scenario-based topics.

01

DEVOPS FUNDAMENTALS

Q1

What is DevOps?

I'd describe DevOps as a culture and engineering discipline that breaks down the wall between development and operations teams. It's not just tooling -- it's a shift in how we work: developers and ops engineers share ownership of the full lifecycle, from writing code to running it in production. In practice, that means automating the build-test-deploy pipeline, monitoring systems continuously, and treating infrastructure as code so that releases become fast, repeatable, and low-risk instead of rare, manual, and stressful.

Q2

What are the key principles of DevOps?

I usually frame it around five pillars: collaboration between teams that used to work in silos; automation of everything that's repeatable, from builds to provisioning; continuous integration and deployment so code moves to production in small, frequent batches; monitoring and feedback loops that tell us how a release is actually behaving; and security and compliance baked in from the start rather than bolted on at the end. When all five are in place, you get faster releases without sacrificing stability.

KEY POINTS TO MENTION

- Collaboration
- Automation
- Continuous Integration & Deployment
- Monitoring & Feedback
- Security & Compliance

Q3

How does DevOps differ from Agile?

Agile is primarily about how the development team plans and builds software -- sprints, backlogs, iterative delivery. DevOps picks up from there and extends that same iterative philosophy across the operations side: deployment, infrastructure, monitoring, and incident response. I like to put it this way -- Agile gets code built quickly, DevOps gets it delivered and kept running reliably. They're complementary rather than competing approaches.

Q4**What are the key benefits of DevOps?**

From what I've seen on real projects, the benefits show up in a few concrete ways: releases go out faster because the pipeline is automated end to end; collaboration improves because devs, QA, and ops are working off the same shared tooling and metrics; systems scale more efficiently because infrastructure is codified and repeatable; and overall reliability goes up because monitoring and automated rollback catch problems early instead of after they've impacted users.

KEY POINTS TO MENTION

- Faster software releases
- Improved collaboration
- Higher efficiency and scalability
- Better system reliability

Q5**What are the key DevOps tools?**

My day-to-day toolbox usually breaks down by function. For CI/CD I reach for Jenkins, GitLab CI, or GitHub Actions depending on the client's ecosystem. For containerization it's Docker, occasionally Podman where rootless containers matter. For orchestration, Kubernetes is the default, with OpenShift showing up in more regulated enterprise environments. Monitoring is Prometheus and Grafana, configuration management is Ansible (and Puppet or Chef on legacy estates), and Git underpins version control across all of it.

KEY POINTS TO MENTION

- CI/CD: Jenkins, GitLab CI, GitHub Actions
- Containerization: Docker, Podman
- Orchestration: Kubernetes, OpenShift
- Monitoring: Prometheus, Grafana
- Configuration Management: Ansible, Puppet, Chef
- Version Control: Git

02

VERSION CONTROL & GIT

Q6

What is Git?

Git is a distributed version control system -- every developer has a full copy of the repository history locally, not just a snapshot from a central server. That's what makes it fast and resilient: I can commit, branch, and inspect history offline, and syncing with a remote is just another Git operation. It's the backbone of almost every CI/CD pipeline I've built, since the pipeline typically triggers off a push or merge event.

Q7

Difference between Git and GitHub/GitLab?

Git itself is the version control engine -- it manages commits, branches, and history on your machine. GitHub and GitLab are hosted platforms built on top of Git that add a web UI, access control, issue tracking, pull/merge requests, and CI/CD runners. So Git is the tool, and GitHub/GitLab are the collaboration layer and automation platform around it.

Q8

What is the difference between git pull and git fetch?

git fetch pulls down the latest commits and branches from the remote but doesn't touch my working branch -- it just updates my local view of the remote. git pull does that same fetch and then immediately merges those changes into my current branch. I tend to prefer fetch-then-merge as two explicit steps in team environments, because it lets me review incoming changes before they touch my working copy.

KEY POINTS TO MENTION

- git fetch: Downloads changes but doesn't merge.
- git pull: Fetches and merges changes into the working branch.

Q9

Explain Git branching strategies.

It really depends on the release cadence of the team. For fast-moving product teams I favor feature branching, where each piece of work gets its own short-lived branch off main. For enterprise release trains I've used Gitflow, which formalizes main, develop, feature, release, and hotfix branches for more controlled releases. And for teams doing true continuous delivery, trunk-based development -- small commits merged straight into main behind feature flags -- keeps integration friction to a minimum.

KEY POINTS TO MENTION

- Feature Branching: Develop features in isolated branches.
- Gitflow: Uses main, develop, feature, release, and hotfix branches.
- Trunk-based development: Continuous integration into main.

Q1
0

How do you resolve a merge conflict in Git?

First I run `git status` to see exactly which files are conflicted. Then I open each one and manually resolve the conflict markers, deciding which changes to keep or how to combine them. Once I'm confident the code is correct, I stage the resolved files with `git add` and commit with a message like "Resolved conflict" so the history is clear about what happened. On larger teams I'll also re-run the test suite before pushing, just to be sure the resolution didn't silently break anything.

KEY POINTS TO MENTION

- Identify conflicts using `git status`.
- Manually edit the conflicting files.
- Add (`git add`) and commit (`git commit -m "Resolved conflict"`) the resolved files.

03

CI/CD (JENKINS, GITLAB CI/CD, GITHUB ACTIONS)

Q1
1

What is Continuous Integration (CI)?

CI is the practice of automatically integrating code from multiple developers into a shared repository many times a day, with every merge triggering an automated build and test run. The goal is to catch integration issues within minutes of them being introduced, rather than discovering them weeks later during a painful manual merge.

Q1
2

What is Continuous Deployment (CD)?

CD takes that automated pipeline one step further and automates the actual delivery of software all the way through testing into production, without a manual gate. I distinguish this from Continuous Delivery, where the pipeline is fully automated up to a manual "go" button before production -- CD removes that button entirely once confidence in the pipeline is high enough.

Q1
3

Explain Jenkins Pipeline.

Jenkins gives me two ways to define a pipeline. A Declarative Pipeline uses a structured, opinionated syntax in the Jenkinsfile -- it's easier to read and is what I default to for most teams. A Scripted Pipeline is written in full Groovy and gives much more flexibility for complex logic, at the cost of readability and a steeper learning curve. I'll reach for scripted only when the declarative syntax can't express what I need.

KEY POINTS TO MENTION

- A Declarative Pipeline defines the entire CI/CD process in Jenkinsfile.
- A Scripted Pipeline provides greater flexibility but requires Groovy scripting.

Q1
4

How do you secure Jenkins?

Securing Jenkins is one of those things people skip until an audit forces the issue. I always set up Role-Based Access Control so people only see and trigger the jobs relevant to them, store all secrets in the Jenkins credentials store rather than hardcoding them in job configs, enforce HTTPS on the controller, and keep plugins patched and pruned since outdated plugins are the most common attack surface I've seen exploited.

KEY POINTS TO MENTION

- Use Role-Based Access Control (RBAC).
- Encrypt secrets using Jenkins credentials.
- Use HTTPS and limit plugin vulnerabilities.

Q1
5

What are GitHub Actions and their advantages?

GitHub Actions is GitHub's native CI/CD automation tool, with workflows defined in YAML directly alongside the code. What I like about it is the setup speed -- there's no separate server to provision -- plus the marketplace of pre-built actions saves a lot of boilerplate, and because it's tightly integrated with GitHub events (PRs, issues, releases), building event-driven automation feels very natural.

KEY POINTS TO MENTION

- A CI/CD automation tool integrated with GitHub.
- Advantages: Easy setup, built-in marketplace, YAML-based workflows.

04

CONFIGURATION MANAGEMENT (ANSIBLE, PUPPET, CHEF)

Q1
6

What is Configuration Management in DevOps?

It's the discipline of managing the state of servers and systems through code rather than manual changes, so that every environment -- dev, staging, production -- ends up in a known, consistent, reproducible state. It removes the "it works on my machine" problem because the configuration itself is versioned and applied identically everywhere.

Q1
7

How does Ansible differ from Puppet and Chef?

The way I explain it: Ansible is agentless and push-based -- it connects over SSH and pushes changes out from a control node, using YAML playbooks that are easy to read even for people newer to the tool. Puppet is agent-based and pull-based, with agents periodically checking in and pulling their desired state, defined in Puppet's own DSL. Chef is also agent-based but uses a full Ruby DSL, which gives more programming power but a steeper learning curve. For most greenfield projects I default to Ansible simply because there's no agent to maintain.

KEY POINTS TO MENTION

- Ansible: Agentless, YAML-based, push model.
- Puppet: Agent-based, uses Puppet DSL, pull model.
- Chef: Uses Ruby DSL, agent-based.

Q1
8

What is an Ansible Playbook?

A playbook is a YAML file that describes the desired end state of a system as a sequence of tasks -- install this package, template that config file, restart this service. I write playbooks to be idempotent, meaning I can run the same playbook a hundred times and it only changes what's actually out of sync, which is what makes Ansible safe to re-run in production.

Q1
9

Explain Infrastructure as Code (IaC).

IaC means defining and provisioning infrastructure -- servers, networks, load balancers -- through machine-readable configuration files instead of manual console clicks. Tools like Terraform for provisioning and Ansible for configuration let me version infrastructure changes in Git, review them through pull requests, and apply them consistently across environments, which is exactly the kind of auditable, repeatable process I've built for client cloud migrations.

Q2
0

What is an Ansible role?

A role is Ansible's way of packaging playbooks into a reusable, structured unit -- with standard subdirectories for tasks, handlers, templates, variables, and defaults. Instead of one giant playbook, I break automation into roles like "nginx" or "docker-install" that I can share across projects and compose together, which keeps things modular and much easier to maintain at scale.

05

CONTAINERS & ORCHESTRATION (DOCKER, KUBERNETES)

Q2
1

What is Docker?

Docker is a containerization platform that packages an application together with everything it needs to run -- libraries, runtime, system tools -- into a single portable image. The big advantage I lean on is consistency: the container behaves the same on my laptop, in CI, and in production, because it's the exact same image running in an isolated environment rather than depending on whatever happens to be installed on the host.

Q2
2

What is a Dockerfile?

It's a plain-text script of instructions -- FROM, COPY, RUN, EXPOSE, CMD, and so on -- that Docker reads to build an image layer by layer. I try to keep Dockerfiles lean by using minimal base images and multi-stage builds, which keeps the final image small and reduces the attack surface for vulnerability scanning tools like Trivy to flag.

Q2
3

What is Kubernetes?

Kubernetes is the orchestration platform I use to run and manage containerized applications at scale -- it handles scheduling containers onto nodes, restarting them when they fail, scaling them up or down, and rolling out updates with zero downtime. Most of the EKS migration work I do centers on designing Kubernetes manifests and Helm charts so applications get all of that resilience without manual intervention.

Q2
4

What are Kubernetes Pods?

A Pod is the smallest deployable unit in Kubernetes -- it wraps one or more tightly coupled containers that share the same network namespace and storage volumes. Most commonly it's a single application container, but I'll add a sidecar container to a Pod when I need something like a log shipper or a service mesh proxy running alongside the main app.

Q2
5

What is Helm in Kubernetes?

Helm is the package manager for Kubernetes -- I use it to template and version application manifests into reusable "charts," so deploying a full application with its Deployments, Services, ConfigMaps, and Ingress rules becomes a single `helm install` or `helm upgrade` command instead of applying a dozen raw YAML files by hand. It's also how I manage environment-specific configuration through values files.

06

CLOUD & INFRASTRUCTURE AS CODE (AWS, AZURE, GCP, TERRAFORM)

Q2
6

What is Terraform?

Terraform is an open-source Infrastructure as Code tool that lets me declaratively define cloud resources -- VPCs, EC2 instances, EKS clusters, IAM roles -- in HCL, then plan and apply those changes across AWS, Azure, or GCP through the same workflow. Because it's declarative, I describe the desired end state and Terraform figures out what needs to be created, changed, or destroyed to get there.

Q2
7

What is the difference between Terraform and CloudFormation?

Terraform is cloud-agnostic, so the same tool and workflow apply whether I'm provisioning on AWS, Azure, or GCP, and it maintains its own state file to track resources. CloudFormation is AWS-specific but has the advantage of being natively integrated with AWS services, sometimes supporting new AWS features slightly before Terraform's provider catches up. On multi-cloud or hybrid engagements I default to Terraform; on AWS-only shops with heavy AWS-native tooling, CloudFormation is a reasonable choice too.

KEY POINTS TO MENTION

- Terraform: Multi-cloud, state management, declarative.
- CloudFormation: AWS-specific, integrated with AWS services.

Q2
8

Explain the Terraform state file.

The state file is Terraform's record of what infrastructure it has actually created and how those resources map to the configuration. It's what lets Terraform calculate a diff on the next plan instead of blindly re-creating everything. In any real project I store this remotely -- typically in an S3 bucket with a DynamoDB table for locking -- rather than leaving it on a local disk where a teammate could easily cause a conflicting or lost state.

Q2
9

What is an AWS IAM role?

An IAM role is a set of permissions that can be assumed by an AWS service, application, or user without needing long-lived credentials. I use roles constantly for things like letting an EKS pod assume a role to access S3, or letting a CI/CD pipeline assume a deployment role -- it's the core mechanism for enforcing least-privilege access across the accounts I manage.

Q3
0

What is auto-scaling in AWS?

Auto-scaling automatically adjusts the number of running instances -- or pods, in the Kubernetes case -- based on real-time demand signals like CPU, memory, or custom CloudWatch metrics. On the infrastructure side I configure Auto Scaling Groups so EC2 capacity grows during traffic spikes and shrinks back down afterward, which keeps performance steady while controlling cost.

07

MONITORING & LOGGING (PROMETHEUS, GRAFANA, ELK STACK)

Q3
1

What is Prometheus?

Prometheus is an open-source monitoring system built around time-series data -- it scrapes metrics from targets at regular intervals and stores them so I can query trends over time. I pair it with alerting rules so the moment a metric like error rate or latency crosses a threshold, the team gets notified before customers do.

Q3
2

What is Grafana used for?

Grafana is the visualization layer I put on top of Prometheus, and often other data sources too -- it turns raw metrics into dashboards that make trends, spikes, and anomalies immediately obvious. I usually build separate dashboards for infrastructure health, application performance, and business KPIs, so different stakeholders can look at exactly what matters to them.

Q3
3

Explain the ELK Stack.

ELK covers the log pipeline end to end. Logstash (or Fluentd in some of my setups) ingests and parses logs from various sources, Elasticsearch indexes and stores that data so it's searchable at scale, and Kibana provides the visualization and search interface on top. It's what I reach for when metrics alone don't explain what happened and I need to search actual log lines across a distributed system.

KEY POINTS TO MENTION

- Elasticsearch: Search engine
- Logstash: Log ingestion
- Kibana: Visualization

Q3
4

What is observability?

Observability is the broader capability of understanding what's happening inside a system just by examining its outputs -- logs, metrics, and traces together. It's a step beyond traditional monitoring: monitoring tells me a known failure mode has occurred, observability lets me investigate and answer questions about failure modes I didn't anticipate in advance.

Q3
5

What are service-level objectives (SLOs)?

SLOs are the specific, measurable targets I set for a service's performance and availability -- for example, 99.9% of requests succeed within 300ms over a rolling 30-day window. They give the team an objective way to decide whether reliability work or new features should be the priority in a given sprint, based on how much of the error budget has been consumed.

08

SECURITY IN DEVOPS (DEVSECOPS)

Q3
6

What is DevSecOps?

DevSecOps is about weaving security checks directly into the DevOps pipeline rather than treating security as a separate gate at the end. In the pipelines I build, that means static code scanning, dependency and container image scanning, and policy checks run automatically on every commit, so vulnerabilities get caught in minutes, not during a pre-release audit weeks later.

Q3
7

What is OWASP?

OWASP -- the Open Web Application Security Project -- is a nonprofit that publishes widely used security guidelines and resources, most notably the OWASP Top 10 list of critical web application risks. I use it as a baseline checklist when reviewing application security posture and when configuring tools like OWASP ZAP for dynamic testing.

Q3
8

How do you secure a containerized environment?

I approach it in layers. At the image level, I build from minimal, trusted base images to shrink the attack surface, and scan every image for known vulnerabilities before it's allowed to deploy. At the cluster level, I implement Kubernetes RBAC so workloads and users only have the permissions they actually need, and I combine that with network policies and pod security standards to limit what a compromised container can reach.

KEY POINTS TO MENTION

- Use minimal base images.
- Implement RBAC in Kubernetes.
- Scan images for vulnerabilities.

Q3
9

What is Shift-Left Security?

Shift-left means moving security checks earlier in the development lifecycle -- into the IDE, the pull request, and the CI pipeline -- instead of waiting until a pre-production security review. The earlier I can hand a developer a finding, the cheaper and faster it is to fix, and the less likely it is to become a production incident.

Q4
0

What are secrets management tools?

These are tools purpose-built for storing and dynamically issuing sensitive data like API keys, database credentials, and certificates, instead of hardcoding them anywhere in code or config files. I typically use HashiCorp Vault for centralized secret storage with dynamic credentials, AWS Secrets Manager when I'm fully in the AWS ecosystem, and native Kubernetes Secrets for cluster-scoped values, usually encrypted at rest.

KEY POINTS TO MENTION

- HashiCorp Vault
- AWS Secrets Manager
- Kubernetes Secrets

09

SRE & RELIABILITY ENGINEERING

Q4
1

What is Site Reliability Engineering (SRE)?

SRE is the practice of applying software engineering discipline to operations problems -- treating reliability targets, capacity planning, and incident response as engineering problems with measurable outcomes rather than purely reactive firefighting. It's a mindset I bring into DevOps work even when the formal title isn't SRE, because the same rigor around SLOs and error budgets improves any production system.

Q4
2

What are SLAs, SLOs, and SLIs?

I think of these as three layers of the same commitment. The SLI is the actual measured indicator, like request latency or error rate. The SLO is the internal target I set for that indicator, like 99.9% availability. The SLA is the external, often contractual, promise to the customer -- usually looser than the SLO -- with defined consequences if it's breached.

KEY POINTS TO MENTION

- SLA: Service Level Agreement
- SLO: Service Level Objective
- SLI: Service Level Indicator

Q4
3

What is Chaos Engineering?

It's the practice of deliberately injecting failure -- killing a pod, cutting network access, spiking latency -- into a system to verify it actually behaves the way we assume it will under stress. I've used tools like Chaos Mesh in controlled environments to validate that failover and auto-healing actually work, rather than trusting the architecture diagram.

Q4
4

What is an Error Budget?

The error budget is simply 100% minus the SLO -- the amount of acceptable failure or downtime a service is allowed before it's considered out of compliance. If the SLO is 99.9%, the error budget is 0.1% of requests or time. I like using it as a shared metric between dev and ops: once the budget is burned, the team's priority shifts from shipping new features to stabilizing the service.

Q4
5

What are blameless postmortems?

Blameless postmortems are incident reviews focused entirely on understanding what happened in the system and process -- not on assigning fault to an individual. I've found that when people aren't afraid of being blamed, they're far more honest about what actually went wrong, which surfaces the real root cause instead of a convenient scapegoat.

10

ADVANCED CI/CD CONCEPTS

Q4
6

How do you implement Blue-Green Deployment?

I keep two identical production environments -- Blue, which is currently live, and Green, which receives the new release. I deploy and fully test the new version on Green while Blue continues serving real traffic, and once I'm confident, I switch the router or load balancer to point at Green. If anything goes wrong, rolling back is just flipping traffic back to Blue, which makes it one of the safest deployment patterns I use for critical services.

KEY POINTS TO MENTION

- Maintain two environments (Blue = live, Green = new).
- Switch traffic after testing Green.

Q4
7

What is Canary Deployment?

Canary deployment releases a new version to a small slice of real traffic or users first, then gradually increases that percentage as I watch error rates and latency metrics stay healthy. It limits blast radius -- if something's wrong, only a small fraction of users are affected before I catch it and roll back.

KEY POINTS TO MENTION

- Gradual release to a small subset of users before full rollout.

Q4
8

What is a Rolling Update?

A rolling update replaces old application instances with new ones incrementally, a few at a time, instead of all at once -- which is exactly how Kubernetes Deployments behave by default. It avoids downtime since there's always a mix of healthy old and new pods serving traffic during the transition, though it does mean two versions run side by side briefly, which I account for when designing backward-compatible APIs.

KEY POINTS TO MENTION

- Gradual replacement of old instances with new ones without downtime.

Q4
9

How do you handle secrets in CI/CD pipelines?

I never let secrets live in plain text in a pipeline definition or repo. Instead I inject them at runtime as environment variables from a secure store -- HashiCorp Vault or AWS Secrets Manager -- so the actual credential value never gets written to logs or version control, and I scope each pipeline's access to only the secrets it genuinely needs.

KEY POINTS TO MENTION

- Use environment variables, HashiCorp Vault, or AWS Secrets Manager.

Q5
0

How do you prevent deployment failures?

It comes down to layering safety nets: solid automated test coverage catches regressions before merge, a well-defined rollback strategy means I can revert quickly if something slips through, and feature flags let me decouple deploying code from actually releasing a feature to users, so I can turn a risky change off instantly without a redeploy.

KEY POINTS TO MENTION

- Implement automated testing, rollback strategies, and feature flags.

11

GITOPS & INFRASTRUCTURE AUTOMATION

Q5
1

What is GitOps?

GitOps is an operating model where Git is the single source of truth for both application and infrastructure state -- every change goes through a pull request, and a tool like ArgoCD continuously reconciles the live cluster to match what's declared in the repo. I've built platforms around this model because it gives a full audit trail, easy rollback via git revert, and removes the need for engineers to have direct kubectl access to production.

KEY POINTS TO MENTION

- A DevOps model where infrastructure changes are managed via Git repositories.

Q5
2

How does GitOps differ from traditional IaC?

Traditional IaC is usually push-based -- I run terraform apply or a pipeline job to push changes out. GitOps flips that: an agent inside the cluster pulls the desired state from Git and continuously reconciles any drift automatically, without a human or pipeline pushing changes directly. That continuous reconciliation is the key difference -- it's self-healing by design.

KEY POINTS TO MENTION

- GitOps enforces version-controlled infrastructure and automatic reconciliation.

Q5
3

What are the best practices for Terraform state management?

I always store state in a remote backend like an S3 bucket, never on a local machine, so the whole team works off the same source of truth. I pair that with state locking -- typically via a DynamoDB table -- so two people can't run apply simultaneously and corrupt the state. I also separate state by environment and workspace so a mistake in staging can't bleed into production.

KEY POINTS TO MENTION

- Store state in remote backends (S3, Azure Blob).
- Use state locking to prevent conflicts.

Q5
4

What is a Terraform module?

A module is a self-contained, reusable, parameterized bundle of Terraform configuration -- for example, a "vpc" module that takes CIDR ranges as inputs and outputs subnet IDs. I build a library of internal modules across client engagements so common patterns like VPCs, EKS clusters, or RDS instances are defined once and consumed consistently everywhere instead of copy-pasted.

KEY POINTS TO MENTION

- A reusable, parameterized collection of Terraform configurations.

Q5
5

What is drift detection in Terraform?

Drift happens when the real infrastructure diverges from what's recorded in Terraform state -- usually because someone made a manual change in the console. I run scheduled terraform plan jobs specifically to detect that drift, so it surfaces as a diff I can review and reconcile rather than silently causing unexpected behavior on the next apply.

KEY POINTS TO MENTION

- Detecting changes in infrastructure that are not in the Terraform state.

1 2

KUBERNETES ADVANCED TOPICS

Q5
6

What is a Kubernetes DaemonSet?

A DaemonSet guarantees that a copy of a specific pod runs on every node in the cluster (or a selected subset). I use this pattern for node-level agents that need to be everywhere -- log collectors, monitoring exporters like the Prometheus Node Exporter, or CNI networking components.

KEY POINTS TO MENTION

- Ensures a pod runs on every node in a cluster.

Q5
7

What is a Kubernetes StatefulSet?

StatefulSets are for workloads that need stable, predictable identity and storage across restarts -- things like databases or message queues. Unlike a Deployment, each pod gets a stable network name and its own persistent volume that follows it even if it's rescheduled to a different node, which matters a lot for anything stateful I'm running in-cluster.

KEY POINTS TO MENTION

- Used for stateful applications, providing stable network IDs and persistent storage.

Q5
8

How does Kubernetes Horizontal Pod Autoscaler (HPA) work?

HPA watches metrics -- CPU and memory by default, or custom metrics via the metrics API -- and automatically adjusts the number of pod replicas in a Deployment to match demand. I configure target utilization thresholds so the workload scales out under load and scales back in once traffic drops, which keeps resource spend aligned with actual usage.

KEY POINTS TO MENTION

- Adjusts the number of pods based on CPU/memory metrics.

Q5
9

What is a Kubernetes Ingress?

Ingress is the resource that manages external HTTP/HTTPS access into services running inside the cluster, typically backed by a controller like NGINX or an AWS ALB controller. It gives me a single place to configure routing rules, TLS termination, and host- or path-based routing instead of exposing every service through its own load balancer.

KEY POINTS TO MENTION

- A resource managing external access to services via HTTP/HTTPS.

Q6
0

What is Kubernetes RBAC?

RBAC lets me define exactly which actions a user or service account can perform on which resources in the cluster -- through Roles or ClusterRoles bound to specific identities. It's central to how I enforce least privilege in multi-tenant clusters, so a CI pipeline's service account, for instance, can only deploy to its designated namespace and nothing else.

KEY POINTS TO MENTION

- Role-Based Access Control for managing permissions in a cluster.

1 3

MONITORING & LOGGING ADVANCED CONCEPTS

Q6
1

What is PromQL in Prometheus?

PromQL is Prometheus's query language for slicing and aggregating the time-series metrics it collects -- things like `rate()`, `sum()` by label, or histogram quantiles. I use it constantly to build alerting rules and Grafana panels, for example calculating a rolling error rate or p99 latency directly from raw counters and histograms.

Q6
2

How do you monitor Kubernetes clusters?

My standard stack is the Kubernetes Metrics Server for basic resource metrics feeding the HPA, Prometheus scraping cluster and application metrics via exporters and the kube-state-metrics add-on, and Grafana dashboards on top to visualize node health, pod resource usage, and application-level SLIs in one place.

KEY POINTS TO MENTION

- Use Prometheus, Grafana, and Kubernetes Metrics Server.

Q6
3

How do you centralize logs in a distributed system?

With services spread across many pods and nodes, I can't chase logs manually, so I ship everything into a central pipeline -- either the ELK stack or Fluentd/Fluent Bit forwarding to Elasticsearch or another sink -- so all logs are searchable and correlated in one interface regardless of which pod or node produced them.

KEY POINTS TO MENTION

- Use the ELK Stack or Fluentd for log aggregation.

Q6
4

What is the difference between tracing and logging?

Logging captures discrete, timestamped events -- "user login failed," "cache miss" -- from a single service in isolation. Tracing follows a single request as it hops across multiple services, showing me exactly where time was spent end to end. When I'm debugging a slow request in a microservices architecture, tracing is what actually shows me which downstream call is the bottleneck.

KEY POINTS TO MENTION

- Logging captures discrete events; tracing follows a request's lifecycle.

Q6
5

How does OpenTelemetry help with observability?

OpenTelemetry gives me a single, vendor-neutral standard and set of SDKs for collecting logs, metrics, and traces across services, instead of instrumenting each service differently for each backend. That means I can switch or combine backends -- Prometheus, Jaeger, whatever the client already runs -- without re-instrumenting the application code.

KEY POINTS TO MENTION

- Provides unified telemetry (logs, metrics, traces) across services.

14

SECURITY BEST PRACTICES IN DEVOPS

Q6
6

How do you implement DevSecOps in a pipeline?

I add automated security gates at each stage of the pipeline: SAST scanning the source code on every commit, dependency and container scanning before an image is allowed to be pushed, and DAST tests against a running staging environment before promotion to production. The key is making these checks fast enough that they don't become a bottleneck developers try to bypass.

KEY POINTS TO MENTION

- Integrate security scanning tools (SAST, DAST) into CI/CD.

Q6
7

What is a SAST tool?

SAST -- Static Application Security Testing -- analyzes source code without running it, looking for patterns like SQL injection risks, hardcoded secrets, or insecure library usage. I run SonarQube for code quality and security combined, and Snyk specifically for dependency vulnerability scanning, both wired into the pipeline so findings show up right in the pull request.

KEY POINTS TO MENTION

- Static Application Security Testing (e.g., SonarQube, Snyk).

Q6
8

What is a DAST tool?

DAST -- Dynamic Application Security Testing -- tests a running application from the outside, the way an attacker would, probing for things like injection flaws or misconfigured headers. I typically run OWASP ZAP against a staging environment as part of the pipeline, and reserve tools like Burp Suite for deeper manual penetration testing when a client needs it.

KEY POINTS TO MENTION

- Dynamic Application Security Testing (e.g., OWASP ZAP, Burp Suite).

Q6
9

How do you implement least privilege access in DevOps?

I scope IAM roles and Kubernetes RBAC bindings as narrowly as the workload actually requires -- no wildcard permissions unless there's a genuine operational reason. I combine that with enforced MFA on any human access to production systems, and I periodically review permissions to catch scope creep that accumulates over time.

KEY POINTS TO MENTION

- Use IAM roles, RBAC, and enforce MFA.

Q7
0

What is container image scanning?

It's the automated process of checking a container image's layers and installed packages against known vulnerability databases before that image is trusted to run. I run Trivy directly in the CI pipeline so a build fails automatically if it introduces a critical CVE, which keeps vulnerable images from ever reaching a registry that production pulls from.

KEY POINTS TO MENTION

- Scanning Docker images for vulnerabilities using tools like Trivy or Clair.

Q7
1**What is a serverless architecture?**

Serverless means running application code without provisioning or managing the underlying servers yourself -- the cloud provider handles scaling, patching, and capacity. AWS Lambda is the one I use most, where I deploy a function and it just runs in response to events, with the infrastructure entirely abstracted away from me.

KEY POINTS TO MENTION

- Running applications without managing infrastructure (e.g., AWS Lambda).

Q7
2**What are the benefits of serverless computing?**

The big wins are auto-scaling that happens transparently with zero configuration on my part, cost efficiency since I only pay for actual execution time rather than idle capacity, and a real reduction in operational overhead because there's no OS patching or server fleet to manage.

KEY POINTS TO MENTION

- Auto-scaling, cost efficiency, and reduced operational overhead.

Q7
3**How does Kubernetes compare to AWS Lambda?**

Kubernetes is built for running long-lived, containerized applications where I need fine control over scaling behavior, networking, and resource allocation. Lambda is event-driven and stateless by design, ideal for short bursts of work triggered by an event like an S3 upload or API call. I choose based on the workload shape -- steady-state services go on Kubernetes, spiky event-driven logic goes on Lambda.

KEY POINTS TO MENTION

- Kubernetes runs containerized apps, whereas Lambda is event-driven and serverless.

Q7
4

What is a CloudFormation Stack?

A Stack is a collection of AWS resources that CloudFormation provisions, updates, and deletes together as a single unit, based on a template. It's useful when a client's environment is already deeply invested in AWS-native tooling and wants that single-unit lifecycle management without introducing Terraform's separate state file.

KEY POINTS TO MENTION

- A collection of AWS resources managed as a single unit.

Q7
5

What is an API Gateway in cloud environments?

An API Gateway is a managed front door for APIs -- it handles routing requests to backend services, enforcing authentication and rate limits, and collecting monitoring data on every call. I use AWS API Gateway in front of Lambda functions and microservices so I get throttling, request validation, and usage metrics without building that logic into every service myself.

KEY POINTS TO MENTION

- A managed service for routing, securing, and monitoring API requests.

16

ADVANCED NETWORKING & SECURITY

Q7
6

What is a Service Mesh?

A service mesh is a dedicated infrastructure layer, usually implemented with sidecar proxies, that handles service-to-service communication concerns -- traffic routing, retries, mutual TLS, and observability -- outside of the application code itself. I've used Istio on larger microservices deployments where the number of services made manually handling those cross-cutting concerns impractical.

KEY POINTS TO MENTION

- A dedicated infrastructure layer for managing service-to-service communication (e.g., Istio, Linkerd).

Q7
7

How do you secure microservices communication?

I encrypt service-to-service traffic with TLS, put an API gateway in front of externally facing services to centralize auth and rate limiting, and where a service mesh is in play, enable mutual TLS so both sides of every connection verify each other's identity automatically, without each service team having to implement it individually.

KEY POINTS TO MENTION

- Use TLS encryption, API gateways, and mutual TLS authentication.

Q7
8

What is a Zero Trust security model?

Zero Trust assumes no user, device, or service is inherently trusted, even inside the corporate network perimeter -- every request has to be authenticated and authorized on its own merits. It's the principle behind why I push for things like mutual TLS between services and strict IAM scoping rather than relying on network location alone as a security boundary.

KEY POINTS TO MENTION

- A model where no one is trusted by default, requiring strict identity verification.

Q7
9

What is a WAF (Web Application Firewall)?

A WAF sits in front of a web application and filters malicious traffic -- SQL injection attempts, cross-site scripting payloads, known bad request patterns -- before it ever reaches the application. I typically deploy AWS WAF in front of an ALB or CloudFront distribution as an additional layer beyond application-level input validation.

KEY POINTS TO MENTION

- Protects applications from web-based threats like SQL injection and XSS.

Q8
0

How do you protect against DDoS attacks?

I layer defenses: a CDN absorbs and caches traffic at the edge so a spike doesn't hit origin servers directly, rate limiting at the API gateway or WAF level throttles abusive request patterns, and for AWS workloads I enable AWS Shield -- or Cloudflare in front of non-AWS origins -- for dedicated volumetric attack mitigation.

KEY POINTS TO MENTION

- Use CDNs, rate limiting, and AWS Shield/Cloudflare protections.

17

DEVOPS CULTURE & PROCESSES

Q8
1

How do you implement DevOps in a large enterprise?

I usually roll it out in stages rather than a big-bang rewrite. Start with CI/CD adoption on one or two pilot teams to prove the value, then layer in Infrastructure as Code so environments become reproducible, add monitoring and observability so teams can actually see the impact, and finally embed DevSecOps practices once the automation foundation is solid. Trying to do all of it at once in a large org almost always stalls.

KEY POINTS TO MENTION

- Start with CI/CD adoption, IaC, monitoring, and DevSecOps practices.

Q8
2

What are key DevOps KPIs?

The metrics I track most closely are deployment frequency -- how often we ship to production -- mean time to recover (MTTR), which measures how fast we bounce back from an incident, and change failure rate, the percentage of deployments that cause a production issue. Together they give a much more honest picture of engineering health than just "how many features shipped."

KEY POINTS TO MENTION

- Deployment frequency, mean time to recover (MTTR), change failure rate.

Q8
3

How do you handle failures in a DevOps environment?

I make sure there's always a fast rollback path so a bad deploy can be reverted in minutes, run blameless postmortems afterward to understand root cause without pointing fingers, and where the system's maturity allows it, use chaos engineering proactively to find failure modes before they find us in production.

KEY POINTS TO MENTION

- Implement rollback strategies, blameless postmortems, and chaos engineering.

Q8
4

What is a postmortem in DevOps?

A postmortem is a structured retrospective written after an incident -- what happened, the timeline, the root cause, and concrete follow-up actions to prevent recurrence. I treat the action items as the real deliverable; a postmortem that doesn't produce tracked follow-up work tends to just repeat itself.

KEY POINTS TO MENTION

- A retrospective analysis of an incident to prevent recurrence.

Q8
5

What are feature flags?

Feature flags let me toggle a feature on or off in production through configuration, completely decoupled from a code deployment. That separation is powerful -- I can deploy code dark, enable it for a small percentage of users to validate it, and instantly disable it if something looks wrong, without needing a redeploy or rollback.

KEY POINTS TO MENTION

- A mechanism for toggling features on/off in production.

18

PERFORMANCE OPTIMIZATION

Q8
6

What is a CDN, and why is it used?

A CDN is a globally distributed network of edge servers that caches content closer to end users, so requests don't have to travel all the way back to the origin server every time. I use it to cut latency for static assets and to absorb traffic spikes, which also reduces load on backend infrastructure.

Q8
7

How do you optimize CI/CD pipeline performance?

I look for parallelization opportunities first -- running independent test suites or build stages concurrently instead of sequentially -- then add caching for dependencies and build artifacts so repeated runs don't redo unnecessary work. On larger pipelines I also introduce selective testing, running only the tests actually affected by a given change, which can cut feedback time dramatically.

KEY POINTS TO MENTION

- Use parallel builds, caching, and selective testing.

Q8
8

How do you optimize database performance in DevOps?

From the ops side, the levers I usually pull are proper indexing on frequently queried columns, introducing a caching layer like Redis in front of hot read paths, and partitioning large tables so queries don't have to scan the entire dataset. I also make sure monitoring surfaces slow-query logs so problems get caught before they become an outage.

KEY POINTS TO MENTION

- Indexing, caching, and database partitioning.

Q8
9

What is a sidecar pattern in microservices?

The sidecar pattern deploys a secondary, auxiliary container alongside the main application container within the same Pod, handling a cross-cutting concern like logging, monitoring, or a service mesh proxy. It keeps that concern out of the application's own code while still giving it the tight coupling of sharing the pod's network and lifecycle.

KEY POINTS TO MENTION

- Deploying an auxiliary container alongside the main service for logging, monitoring, or security.

Q9
0

How do you reduce cloud costs in a DevOps environment?

I combine auto-scaling so we're not paying for idle capacity, spot instances for fault-tolerant or batch workloads where interruption is acceptable, and cost monitoring tools like AWS Cost Explorer or tagging-based dashboards so the team has visibility into spend by service and can catch runaway costs early rather than at month-end.

KEY POINTS TO MENTION

- Use auto-scaling, spot instances, and cost monitoring tools.

19

INCIDENT MANAGEMENT & DISASTER RECOVERY

Q9
1

What is a runbook in DevOps?

A runbook is a documented, predefined set of steps for handling a specific, known type of incident -- for example, what to check and do when a database connection pool is exhausted. Having runbooks means the person on call at 3 a.m. doesn't have to reason from first principles under pressure; they follow a tested procedure.

KEY POINTS TO MENTION

- A predefined set of procedures for handling incidents.

Q9
2

What is a playbook in incident response?

A playbook is similar in spirit to a runbook but broader in scope -- it's a detailed action plan covering roles, communication steps, and escalation paths for mitigating a security or major system incident, not just the technical remediation steps. I make sure playbooks specify who owns communication to stakeholders, not just what commands to run.

KEY POINTS TO MENTION

- A detailed action plan for mitigating security or system issues.

Q9
3

How do you handle rollback in Kubernetes?

Kubernetes keeps a revision history for Deployments, so if a rollout introduces a problem, I can revert to the last known-good version with `kubectl rollout undo`, and Kubernetes handles rolling the pods back the same way it rolled them forward -- incrementally, without downtime.

KEY POINTS TO MENTION

- Use `kubectl rollout undo` to revert to the previous deployment.

Q9
4

What is RTO and RPO in disaster recovery?

RTO -- Recovery Time Objective -- is how long we can tolerate a service being down before it must be restored. RPO -- Recovery Point Objective -- is how much data loss, measured in time, is acceptable, which directly drives how frequently I need to take backups or replicate data. Both numbers come from the business, and my job is to design a DR architecture that actually meets them.

KEY POINTS TO MENTION

- RTO: Recovery Time Objective (time to restore services).
- RPO: Recovery Point Objective (maximum acceptable data loss).

Q9
5

How do you test disaster recovery in DevOps?

I don't consider a DR plan real until it's been tested -- documentation alone doesn't prove anything works. I run scheduled failover tests, simulating a region or availability zone outage, and time how long recovery actually takes against the target RTO, then feed any gaps back into the plan.

KEY POINTS TO MENTION

- Conduct failover tests and simulate outages.

20

MISCELLANEOUS & FUTURE TRENDS

Q9
6

What is FinOps in cloud computing?

FinOps is the practice of bringing financial accountability into cloud spend the same way we brought engineering discipline into operations -- giving engineering, finance, and business teams shared visibility into cost so decisions about scaling or architecture factor in cost, not just performance. I've worked cost-optimization efforts that were really FinOps in practice, even before the client used that label.

KEY POINTS TO MENTION

- Financial operations to optimize cloud spending.

Q9
7

What is Chaos Engineering?

As I mentioned earlier with resilience testing, it's the deliberate practice of injecting controlled failure into a system to verify it degrades gracefully and recovers as designed, rather than assuming the architecture works based on a diagram alone.

KEY POINTS TO MENTION

- Deliberate testing of system failures to improve resilience.

Q9
8

What is Policy-as-Code?

Policy-as-Code means defining security, compliance, and governance rules in a machine-readable format so they can be automatically enforced and version-controlled, the same way we treat infrastructure. I use tools like Open Policy Agent (OPA) to enforce Kubernetes admission policies, and AWS Service Control Policies at the organization level to guardrail what accounts are allowed to do.

KEY POINTS TO MENTION

- Defining security and compliance policies in code (e.g., OPA, AWS SCPs).

Q9
9

What is AIOps?

AIOps applies machine learning to operations data -- logs, metrics, traces -- to automate anomaly detection, correlate related alerts, and in some cases suggest or trigger remediation, cutting down the noise that on-call engineers have to sift through manually.

KEY POINTS TO MENTION

- AI-driven operations that automate incident detection and resolution.

Q1
00

What are emerging trends in DevOps for 2025?

The direction I'm seeing across client work is more AI-driven automation showing up inside the pipeline itself, wider GitOps adoption as the default deployment model rather than the exception, tighter security built into Kubernetes by default rather than as an afterthought, and continued investment in observability so teams can reason about increasingly distributed systems with confidence.

KEY POINTS TO MENTION

- AI-driven automation, GitOps adoption, enhanced Kubernetes security, and observability improvements.

Closing Note

These 100 questions cover the ground I'd expect in a real DevOps engineer interview loop — from fundamentals through GitOps, Kubernetes internals, security, and reliability engineering. The answers are written the way I'd actually say them out loud, so treat them as a starting point: adapt the specifics to your own project experience and the tools your target company uses.

Compiled and elaborated by Arvind Verma.